LAB 06 — D3.js: Multi-line COVID-19 Chart

International University · Data Science and Data Visualization
Pairs · 150 minutes including 15–20 min introduction · D3 v7

> **Your task:** Build an interactive **four-country line chart** showing historical **cumulative confirmed COVID-19 cases** from **1 April through 1 May 2020 (inclusive)**. A pointer readout must show the values of **all four countries on the same date**, and legend hover must focus one country's line.

Learning goals: transform and group time-series data; draw multi-line paths with scales/axes; explain, test and adapt interactive behavior.

1. What you are building

Source data	Output
`Country/Region`, date columns such as `4/1/20`	Four series: **Vietnam, US, France, Italy**
One or multiple source rows per country	**Country-level daily totals**, not individual province lines
Dates 1 Apr–1 May 2020	X-axis: time (daily); Y-axis: **cumulative confirmed cases**
Mouse and legend events	Shared date readout, four values, legend focus, color legend

CSV source: JHU CSSE — confirmed global time series. This is historical data. The original lab calls for a **multi-line chart**; the **stacked area** is an optional extension in Section 4.

2. Start here (first 10 minutes)

1. Before class, run `examples/01_worked_multiline.html` and `examples/02_worked_stacked_area.html` from the pre-lab; answer its four questions.
2. Create `index.html`, `script.js`, `style.css`, `NOTES.md`. Load **D3 v7 only** and serve locally with `python -m http.server 8000`.
3. Inspect the source before plotting. Start `script.js` with:

```
const URL = 'https://raw.githubusercontent.com/CSSEGISandData/COVID-19/master/' +
  'csse_covid_19_data/csse_covid_19_time_series/' +
  'time_series_covid19_confirmed_global.csv';
d3.csv(URL).then(rows => {
  console.log('Headers:', rows.columns.slice(0, 7));
  console.log('Both dates?', rows.columns.includes('4/1/20'),
                            rows.columns.includes('5/1/20'));
  console.table(rows.filter(r =>
    ['Vietnam', 'US', 'France', 'Italy'].includes(r['Country/Region'])
  ).slice(0, 8));
}).catch(console.error);
```

THINK (AI off, 5 min): Why might France appear in several CSV rows? What breaks if dates are sorted as plain text? Should the Y-axis show cases per day or cases accumulated up to that day? Discuss before implementing; AI is permitted afterwards, but test its suggestions.

3. Build — finish one checkpoint before the next

A. Clean and reshape. Inspect field names. Keep four country names and date headers from `4/1/20` **to** `5/1/20` **inclusive** (31 dates). Convert strings into nonnegative numeric counts; detect blanks/invalid entries **before** `Number(...)` because `Number('')` becomes zero. Group source rows by country; **sum all province/region rows per country per date**. Transform wide CSV into records like:

```
{ country: 'Vietnam', date: new Date(2020, 3, 1), cases: 123 }
```

The record above uses a **fictional 123** only to illustrate the *shape*; **calculate actual values from the CSV**, do not hard-code. Sort dates ascending. For D3 v7 use `d3.group(...)`, **not the obsolete** `d3.nest(...)`

shown in the original lab. **Check:** exactly **4 series × 31 dates = 124** country-date observations; log four values on the first and last days.

B. Draw the four lines. Use `scaleTime()` for dates and `scaleLinear()` for counts; add margins, axes, labels/units, chart title and source/date note. Use `d3.line()` and **one `<path>` per country**, with stable colors. Show a readable legend. **Check:** four paths; dates increase left→right; values increase upward; end values match your computed data. There should be **no paths for each province**.

C. Add interaction. On pointer move across the plotting area, find the nearest date and show **Vietnam, US, France, Italy values together** (in a readout or tooltip), plus a shared vertical guide. Restore/hide on leave. On **legend hover**, emphasize the matching country's line and dim the others; restore all on leave. **Check:** the same date is used for all four values; the legend does not accidentally delete data.

4. Verify and adapt

Record **expected → actual** in `NOTES.md`:

Test	Expected
V1 — reshape	4 countries × 31 dates; each country's provinces summed once per day
V2 — axes/path	4 lines, correct chronological order, case counts nonnegative
V3 — pointer	Same nearest date; four values match the cleaned data
V4 — legend	Hover isolates visually; leave restores all four lines

Bug challenge: Find and fix one failure (e.g., forgetting to aggregate France, comparing date strings, using `d3.nest`, or pointer readout mixing dates). Record failure → cause → fix → retest.

ADAPT (required): Add a **click-to-lock legend focus**: click a country name to keep that line emphasized after moving the pointer away; click it again or choose another country to change the focus; include a way to restore all four lines. Explain the difference between temporary hover and locked selection.

Optional stacked area (title extension): Build a **separate** stacked area showing the four countries' **daily new cases**. Calculate daily change from cumulative counts, document handling of negative revisions, align dates and use `d3.stack()` + `d3.area()`. Label the Y-axis **daily new cases (stacked total)**. Do not mistake cumulative counts for daily incident cases.

5. Submit

One ZIP per pair: `index.html`, `script.js`, `style.css`, `NOTES.md`, and local CSV if used. `NOTES.md`: names/IDs; transformed-data evidence; THINK answers; V1–V4 results; bug/fix; click-lock explanation; brief AI-use disclosure. Be ready to explain the grouping and shared-date readout.

Required page metadata: name, student ID, course title, lab title, and original source statement: “This is all my own work. I did not copy the code from any other source.” Follow institutional academic-integrity rules and accurately disclose external or AI-generated assistance.

Read: Scott Murray, Interactive Data Visualization for the Web, Chapter 11 (Using Paths). The reading demonstrates older D3 syntax; use D3 v7 in your implementation.